

Triplox Query Experiments

Five storage and execution techniques from DuckDB, Datomic and RedisGraph. Each one is explained from first principles, built into Triplox end to end, and measured on one benchmark.

Branches: `bench/harness`, `exp/vectorized-join`, `exp/datom-segments`, `exp/columnar-storage`, `exp/sparse-matrix`, `exp/zone-maps`, `exp/combo-vec-seg`, `exp/combo-vec-col`, `exp/combo-full`. Nothing is pushed.

Five techniques from other database engines were built into Triplox and measured on one benchmark: a vectorized join, many datoms per storage key, columnar segments, sparse adjacency matrices, and zone maps. The reference query throughout is the triangle query: find every three vertices a, b, c with edges $a \rightarrow b$, $b \rightarrow c$ and $a \rightarrow c$, the standard test of a graph join. A profile of it put 55% of its time in the join's row vectors and 28% in index iteration. The vectorized join removes the first cost. Many datoms per key remove most of the second. Adjacency matrices turn graph traversals into array operations and run the triangle query approximately 30 times faster. RedisGraph counts paths with one counter per vertex instead of listing them. Built the same way on top of the matrices, that takes the three-hop count from 6.8 s to under a millisecond. Zone maps only pay for time-travel queries. Three combinations of the techniques were then measured, and two latent defects in the current code were found on the way.

1 The benchmark and the current engine

1.1 The harness

The benchmark is a Cargo bench on the `bench/harness` branch, run as `cargo bench --bench datalog_bench`. It starts an in-memory node, loads a seeded random graph through the normal transaction path, and times ten Datalog queries through the public `db.query()` call. SlateDB runs on memory with no object store. The figures isolate engine and layout costs and say nothing about S3 latency. Against object storage, each key fetch becomes a block read, and the per-key costs in chapters 3 and 4 grow.

An environment variable guards each experiment, and one binary runs both arms of an A/B comparison. Every ratio in this report is on to off within one run of the same binary. The baseline column of the query table was measured earlier and separately, and no ratio uses it.

1.2 Measurement

Every chart and table reports the median of 20 samples per arm and query. The samples come from four passes of five iterations, with the arms interleaved within each pass, on an otherwise idle machine. Confidence intervals on medians and on ratios of medians are bootstrap percentile intervals from 4000 resamples. The p-value comes from a two-sided Mann-Whitney U test between the samples of the two arms. A ratio is printed in colour only if its interval excludes 1. If the interval includes 1, the report treats the difference as no effect, whatever the point estimate. The first iteration of each query in a process is a cold run. It is included, and it widens the intervals on the sub-millisecond queries.

1.3 Input

The benchmark graph is a directed $G(n, p)$ graph with 2000 vertices and $p = 0.01$. It has 39,764 edges. Each vertex is an entity with four attributes. The edges are one cardinality-many reference attribute. Triplox stores the graph the way any application would store it, not as an adjacency list.

This is a narrow workload, and the results must be read with that in mind. The data is uniform and random, with integer keys and values, one string attribute, one reference attribute, and seven of the ten queries

traverse that one attribute. A study of 250,000 real queries from Tableau Public found the opposite shape: half of all stored values are strings, most datasets hold under a thousand tuples with a long tail of huge ones, fewer than 5% of queries contain a join, and query cost sits in string handling, casts and other scalar expressions rather than in joins.¹ This benchmark, like TPC-H, tests none of that. The vectorized join and the segment layouts cut costs that every query pays and should carry over. The adjacency matrices, the algebra and value-based zone maps are shaped for this data. String storage and expression evaluation were not tested at all, and a workload built from real Datomic use would be the fair test.

attribute	type	values	purpose
:g/id	long, unique identity	0 ... 1999	point lookups and lookup refs
:g/to	ref, cardinality many	about 20 targets per vertex	all graph traversals
:g/weight	long	id mod 1000	scalar range predicates and sums
:g/label	string	"label-id"	string equality lookup

1.4 The running example

Two thousand vertices do not fit in a diagram. The rest of the report uses a five-vertex graph with the same schema. Entities are integers, and the letters are only labels. The seven edges contain two triangles: $a \rightarrow b \rightarrow c$ closed by $a \rightarrow c$, and $b \rightarrow c \rightarrow d$ closed by $b \rightarrow d$. Exactly two vertices have a weight above 950.

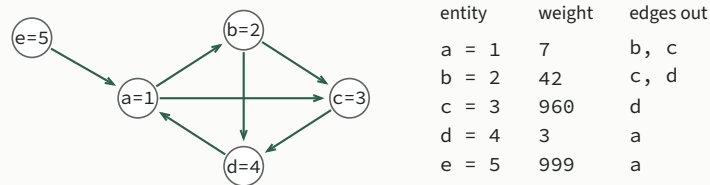
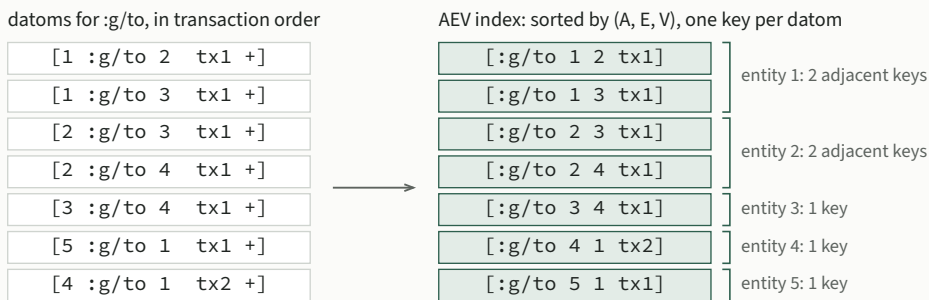


Figure 1: The example graph: seven :g/to edges over five entities. Transaction 1 asserted all edges except $d \rightarrow a$, which transaction 2 asserted.

1.5 How Triplox stores the example

A transaction turns each fact into a *datom*: entity, attribute, value, transaction id, and a flag for assert or retract. Triplox writes every *datom* into six indexes in SlateDB, a log-structured key-value store. Each index holds the same *datoms* sorted in a different component order. Today each *datom* is one SlateDB key in each index. Graph queries use two of the indexes. AEV is sorted by attribute, entity, value and answers “what does entity x point to”. AVE is sorted by attribute, value, entity and answers “who points to y ”.



AVE holds the same seven *datoms* sorted by (A, V, E). Its first two keys are [:g/to 1 4] and [:g/to 1 5]: the two entities that point to 1, again adjacent.

Figure 2: *Datoms* become sorted keys. One entity’s edges form a contiguous key range in AEV. A read of them is one seek and a short range walk. Every optimization in this report changes either the shape of these keys or the way the join reads them.

¹Vogelsgesang et al., “Get Real: How Benchmarks Fail to Represent the Real World”, DBTest 2018.

1.6 How Triplox runs the triangle query

A triangle is three vertices joined by three edges in one direction: $a \rightarrow b$, $b \rightarrow c$, and the closing edge $a \rightarrow c$. The query below asks for every one. It is the standard test of a graph join. The third pattern refers back to the first variable, so the join cannot run as a simple chain. The example graph holds two.

```
[ :find ?a ?b ?c :where [?a :g/to ?b] [?b :g/to ?c] [?a :g/to ?c]]
```

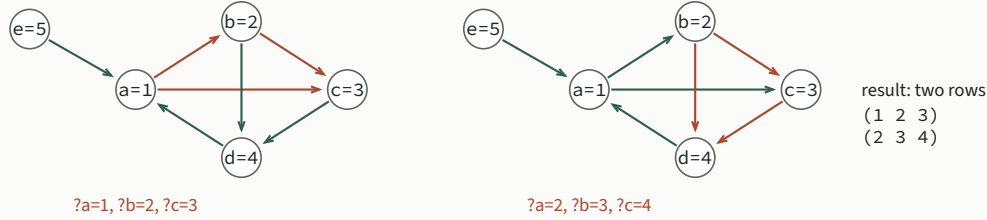


Figure 3: The triangle query on the example graph. Each highlighted set of three edges satisfies all three patterns at once. The edge $d \rightarrow a$ does not close a triangle because no vertex points to both d and a .

The query compiler orders the variables $?a, ?b, ?c$. The *generic join* then extends bindings one variable at a time. At each level it asks every pattern that mentions the variable for candidate values, intersects the candidate sets, and recurses into each survivor. This is a worst-case-optimal join. On the triangle, the third pattern $[?a :g/to ?c]$ prunes $?c$ at once, and the join never materializes the full $a \rightarrow b \rightarrow c$ path list. Every binding at every level performs its own index seek.

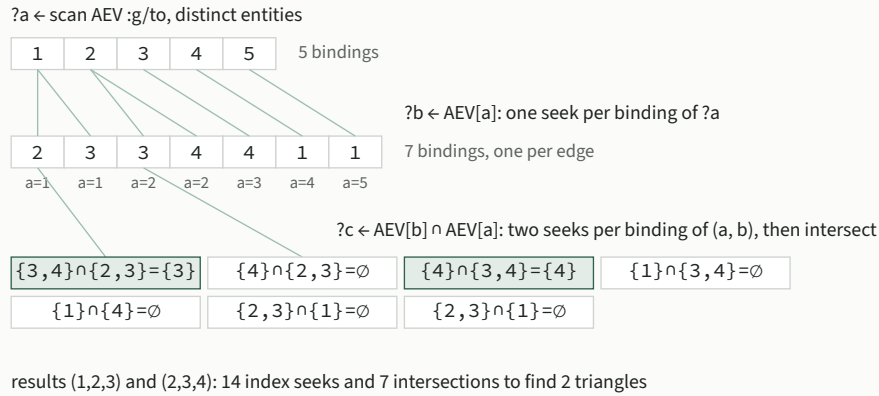


Figure 4: The generic join on the example. Each box at a level is one binding. Each binding costs at least one seek into AEV at the next level. On the benchmark graph the $?b$ level has 39,764 bindings, and each one performs two seeks. Seek count and per-binding bookkeeping dominate the run time.

1.7 Queries

Each query isolates one access path. The row count is the output. Every experiment and configuration compared it against the baseline.

name	Datalog and what it exercises	rows	ms
triangles	<pre>[:find ?a ?b ?c :where [?a :g/to ?b] [?b :g/to ?c] [?a :g/to ?c]]</pre> cyclic three-way join, the case worst-case-optimal joins are designed for	7,821	537
two_hop_count	<pre>[:find (count ?c) :where [?a :g/to ?b] [?b :g/to ?c]]</pre> fan-out over two reference hops, then an aggregate	1	223
three_hop_count	<pre>[:find (count ?d) :where [?a :g/to ?b] [?b :g/to ?c] [?c :g/to ?d]]</pre> three hops, about 16 million intermediate rows before the count	1	7,041
out_degree	<pre>[:find ?a (count ?b) :where [?a :g/to ?b]]</pre> full AEV scan of one attribute, grouped by entity	2,000	44
in_degree_top	<pre>[:find ?b (count ?a) :where [?a :g/to ?b]]</pre> the same scan, grouped by value	2,000	44

name	Datalog and what it exercises	rows	ms
neighbors_of_42	<code>[:find ?b :where [?a :g/id 42] [?a :g/to ?b]]</code> unique-attribute lookup, then one entity's edges	21	1.1
weight_filter	<code>[:find ?e :where [?e :g/weight ?w] [(> ?w 900)]]</code> scalar scan with a range predicate	198	3.7
weight_sum	<code>[:find (sum ?w) :where [?e :g/weight ?w]]</code> single-column aggregate, the nearest thing to an analytical scan	1	3.6
heavy_neighbors	<code>[:find ?a ?b :where [?a :g/to ?b] [?b :g/weight ?w] [(> ?w 950)]]</code> a reference hop joined with a scalar predicate	1,908	68
label_lookup	<code>[:find ?e :where [?e :g/label "label-777"]]</code> string equality through AVE, a control for fixed overhead	1	0.02

1.8 Results overview

One cell per experiment and query, as a speed-up factor. Each chapter shows its own row as a chart of absolute times with every sample, and the appendix holds the medians, intervals and p-values.

	triangles	two_hop	three_hop	out_degree	in_degree	neighbors	weight_filter	weight_sum	heavy_nbrs	label_lookup
Vectorized join (ch. 2)	1.7	1.6	3.5	1.1	1.1	1.0	1.3	1.2	1.3	1.2
Datoms per key (ch. 3)	1.3	1.4	1.1	3.1	3.1	22	2.8	2.9	2.2	1.0
Columnar segments (ch. 4)	1.3	1.4	1.0	3.3	3.3	17	2.8	2.9	2.2	0.8
Adjacency matrices (ch. 5)	29	1.7	1.1	6.9	6.9	45	1.0	1.0	2.6	1.0
Matrix algebra (ch. 5)	30	400	7,700	7.3	7.1	48	1.0	1.0	2.6	1.1
Zone maps (ch. 6)	1.0	1.0	1.0	1.0	1.0	0.8	1.5	1.0	1.1	1.2
Vectorized join + datoms per key (ch. 7)	2.6	3.0	3.9	4.4	4.4	24	5.2	5.2	3.8	1.0
Vectorized join + columnar (ch. 7)	2.5	3.1	3.9	5.1	5.2	18	5.9	5.6	4.1	0.9
Vectorized + columnar + matrices (ch. 7)	49	3.5	3.9	14	13	40	5.8	5.5	6.6	0.9
The same plus matrix algebra (ch. 7)	49	400	7,600	14	13	46	5.9	5.5	6.6	0.9

cell = how many times faster than off. green: faster, darker is larger · red: below 1, slower · grey cell and grey number: no effect, the interval includes 1

Figure 5: Median over median. The chapter 7 rows compare each stack's all-on arm with that stack's own off arm.

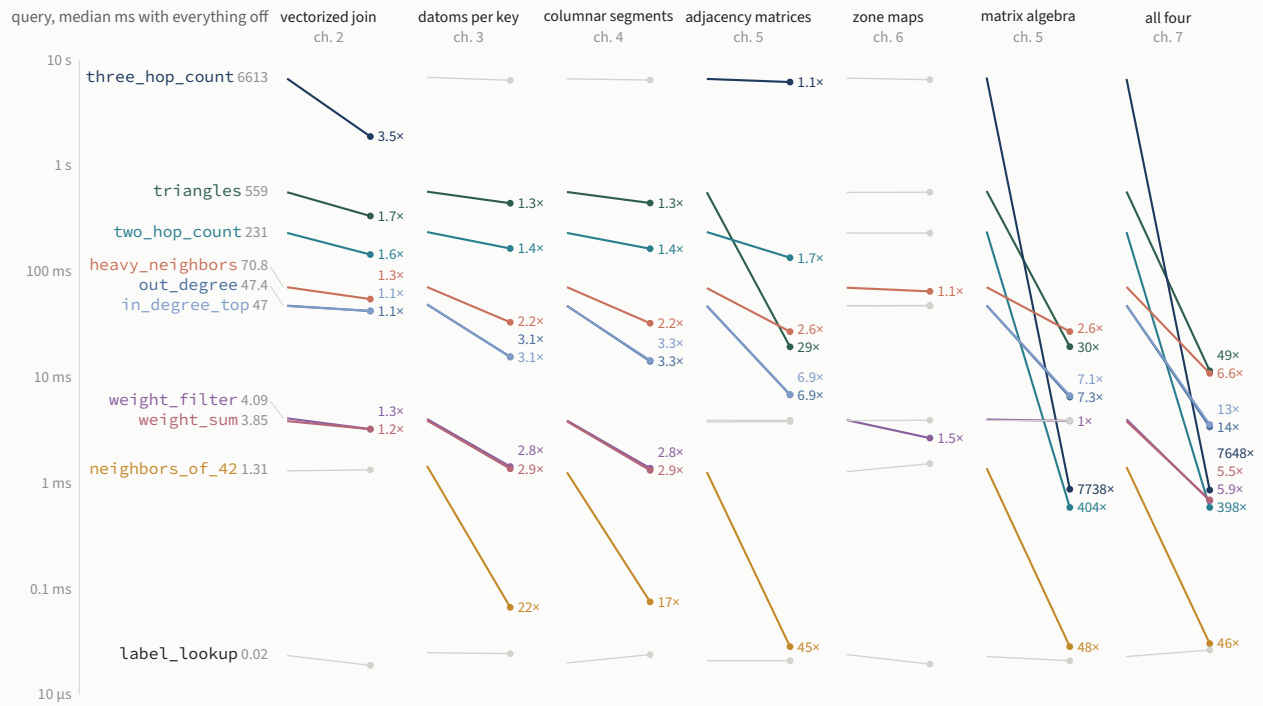


Figure 6: Each line runs from a query's time with everything off to its time with the technique on. Grey: the 95% interval of the ratio includes 1.

2.1 Where the time goes today

The generic join in chapter 1 keeps its intermediate state as rows. Each binding is a `Vec<Bytes>` with one Bytes per bound variable. To extend a level, the join allocates a new row for every parent row, clones the parent's values into it, and appends the new value. The join builds candidate sets the same way, one small vector per candidate, grouped through a `BTreeMap`. None of this is the query's work. It is bookkeeping around the work.

A sampling profile of the triangle query on the benchmark graph shows the effect. The shares below are shares of total query time.

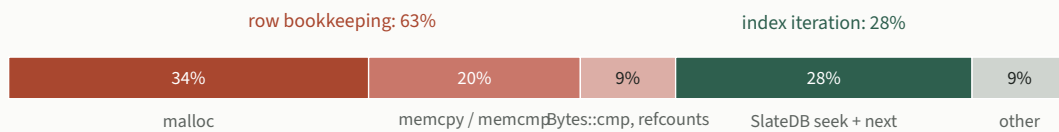


Figure 7: Where the triangle query spends its time before the change, from a 20 second sample of the release binary. Allocation, copying and byte comparison of row vectors take twice as long as reading the index.

2.2 The idea: process columns, not rows

DuckDB follows the MonetDB/X100 design and never hands one row from operator to operator. Each operator takes a *vector* of approximately two thousand values for one column. It does its work in a tight loop over that array and passes the array on. Allocation happens once per vector instead of once per value. The loop body is small, the same instruction runs over adjacent memory, and the compiler keeps the loop in registers.

The second half of the idea is the layout of a set of rows when the query has several variables. Instead of one array per row, keep one array per variable, all of the same length. Row i is then the i -th entry of each array. This layout is called *structure of arrays*. Projection drops an array. An aggregate over one variable reads one contiguous array.

2.3 Mapping onto the generic join

The generic join grows its bindings one variable at a time. Each level of the join is a natural vector: the column of new values for that variable. Each new value belongs to exactly one parent binding at the level above. The new level stores the *index* of its parent instead of a copy of the parent's values. The engine reconstructs a row only when it needs one, by following parent indexes upward.

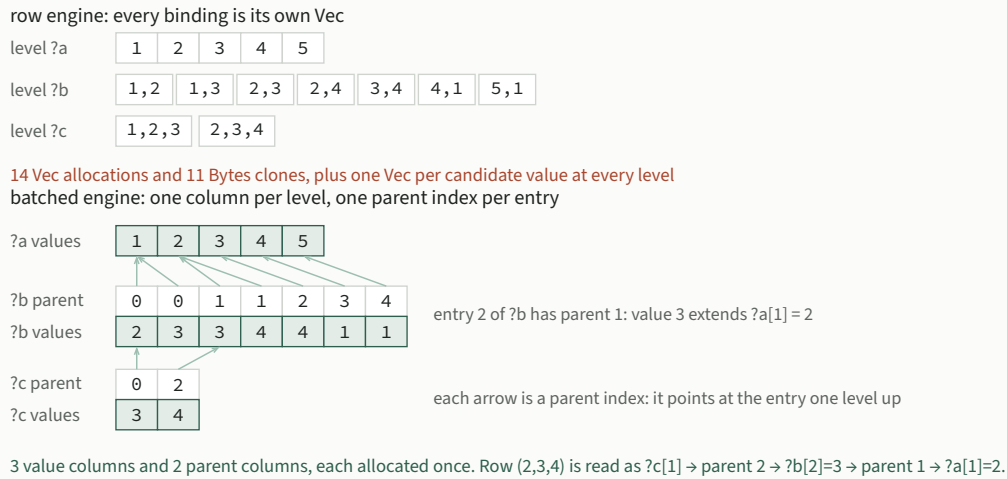


Figure 8: The three levels of the triangle join on the example, in both representations. In the row engine the number of allocations equals the number of bindings. In the batched engine it equals the number of levels.

The implementation adds three parts, and the code and the toggle call the result the batched join. A Batch type holds these columns with structural sharing. A BatchedJoinEngine drives the existing patterns level by level over whole batches. A fast path runs comparison predicates over a column. Grouping for aggregates sorts an array of row indexes instead of building a BTreeMap. Queries with not, or, relation or function clauses go to the existing engine unchanged. The batched engine only sees pure triple patterns, predicates and aggregates.

2.4 Walkthrough: three_hop_count

The query `[ :find (count ?d) :where [?a :g/to ?b] [?b :g/to ?c] [?c :g/to ?d]]` is a chain. No pattern closes a cycle, and no candidate is pruned. The ?d level holds approximately sixteen million entries. The row engine builds sixteen million four-element vectors, and each one carries three cloned Bytes. The batched engine builds one column of sixteen million values and one column of sixteen million parent indexes. The count reads the length of one array. The deeper the join, the larger the share of time that was row bookkeeping, and the larger the gain in the chart below.

2.5 Results

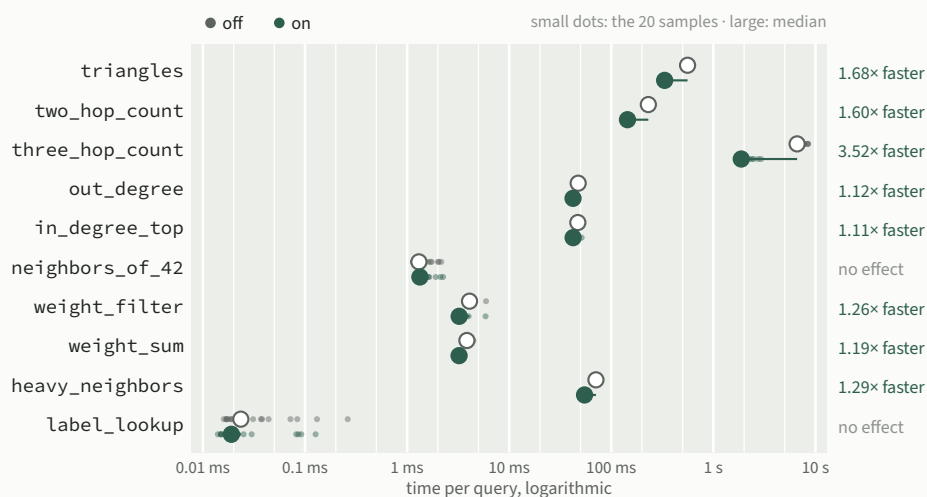


Figure 9: Small dots are the 20 samples of each arm, large markers the medians. “No effect”: the 95% interval of the ratio includes 1.

Correctness Results are identical to the current engine including row order. 642 tests pass with the toggle on and off. Clippy is clean.

Reading the chart	The gain grows with join depth. One level is flat, two levels give approximately 0.6×, and three levels give 0.28×. Single-scan queries are unaffected. The small differences on them are noise.
What is not done	The final level is still fully materialized before an aggregate runs. A columnar aggregate sink would let <code>three_hop_count</code> count without building the <code>id</code> column. The bounded per-level batching from issue 204 is also not implemented. A level is columnar but not chunked, and peak memory is unchanged.
Cost to integrate	Low. No storage format, change feed or planner changes. The risk is the duplicated pattern-validation logic in <code>triple.rs</code> . Only the equivalence test holds the two copies together.

3 Multiple datoms per key exp/datom-segments • TRIPLOX_SEGMENT_SIZE=256

3.1 Where the time goes today

SlateDB is a log-structured merge tree. Keys live in sorted string tables. Each table is split into blocks and has a block index and a Bloom filter. A read of one key finds the table, binary-searches the block index, loads the block, and scans to the key. A range iteration repeats the per-key part for every key: compare the key against the range end, decode the entry, copy it into a Bytes, and advance.

With one datom per key, the `:g/to` attribute is 39,764 keys in AEV. A full scan for `out_degree` performs 39,764 of those steps. Every seek in the triangle join lands on a single 27-byte key.

3.2 The idea: segments

Datomic does not store one datom per storage key. Its indexes are trees. Each leaf is a *segment* of thousands of datoms, stored as one value under one key. The storage layer sees a few thousand large values instead of millions of small ones. A scan pays the per-key cost once per segment and then walks an in-memory array. A point lookup fetches one segment and binary-searches inside it.

The price is write amplification. To add one datom, the writer rewrites the segment that owns it. A segment that grows past its limit splits in two. Datomic accepts this because it batches writes by transaction and rebuilds its indexes in the background.

3.3 Mapping onto SlateDB

This experiment keeps the per-datom encoding as it is and changes only how many datoms share a key. Up to N sorted datom keys are concatenated into one value, each with a length prefix. The SlateDB key of a segment is the key of the *last* datom in it.

SlateDB's API forces that choice. An iterator can seek forward to the first key at or after a target, but it cannot step backward. Suppose the key were the first datom. A seek to a datom in the middle of a segment then lands on the *next* segment and misses it. With segments keyed by their last datom, the seek lands on the segment that owns the target.

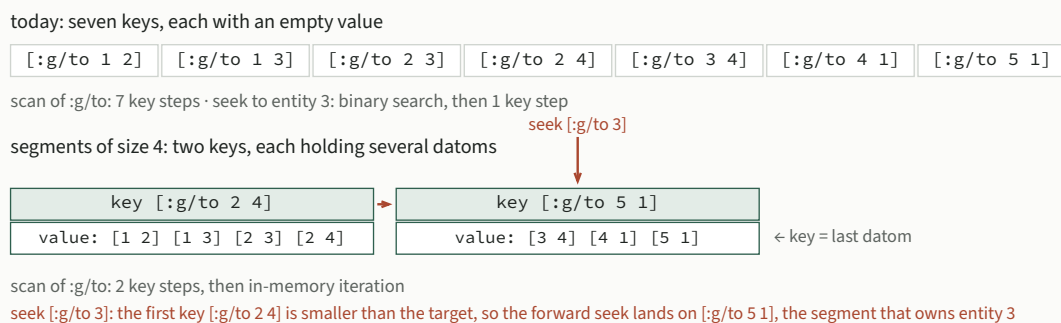


Figure 10: Segments on the example with N = 4. The benchmark uses N = 256, and the AEV keys for `:g/to` drop from 39,764 to 156. A forward seek to any datom returns the segment whose last datom is the first one at or after the target. That segment owns the target.

Reads go through a `KeyCursor` that accepts both layouts in one index. The writer's segment size can change between transactions. Writes merge new datoms into the owning segment. At flush time, a segment that exceeds N splits once.

3.4 Interaction with the incremental engine

The incremental engine does not use the ad-hoc query path. It seeds a DBSP circuit with one EAV scan and then feeds it from the SlateDB change feed, one entry at a time. Today one entry is one datum. With segments, one entry is a whole segment, and a rewrite republishes every datum in it.

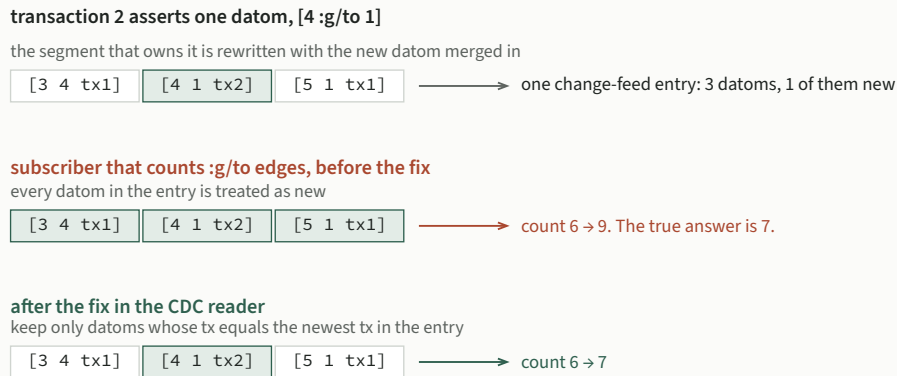


Figure 11: What the change feed carries once several datoms share a key. A segment rewrite republishes the datoms that transaction 1 wrote, and a subscriber that trusts the entry counts them again. The fix filters each datum by its own transaction id. The seeding scan needed the same change. The filter assumes an entry never carries more than one new transaction, which the current write path guarantees but nothing enforces.

The adjacency matrices of chapter 5 and the zone maps of chapter 6 are caches keyed by basis. Neither is updated from the change feed, so each new transaction forces a rebuild on the next query. The vectorized join changes only the ad-hoc engine and has no effect on incremental queries.

3.5 Walkthrough: out_degree and neighbors_of_42

out_degree scans all of :g/to in AEV. Today the scan takes 39,764 iterator steps. With N = 256 it takes 156 steps. Each step returns one value, which the reader decodes into an array and walks in memory. neighbors_of_42 seeks to one entity. Today that is a binary search and about twenty key steps. After the change it is a binary search and one segment decode. These two queries show the largest improvement in the chart.

The triangle join seeks once per binding, 39,764 times at the ?b level. Each seek now decodes a 256-datom segment to read about twenty datoms from it. The per-seek cost rises while the per-key cost falls. The join-heavy queries therefore improve less than the scans.

3.6 Results

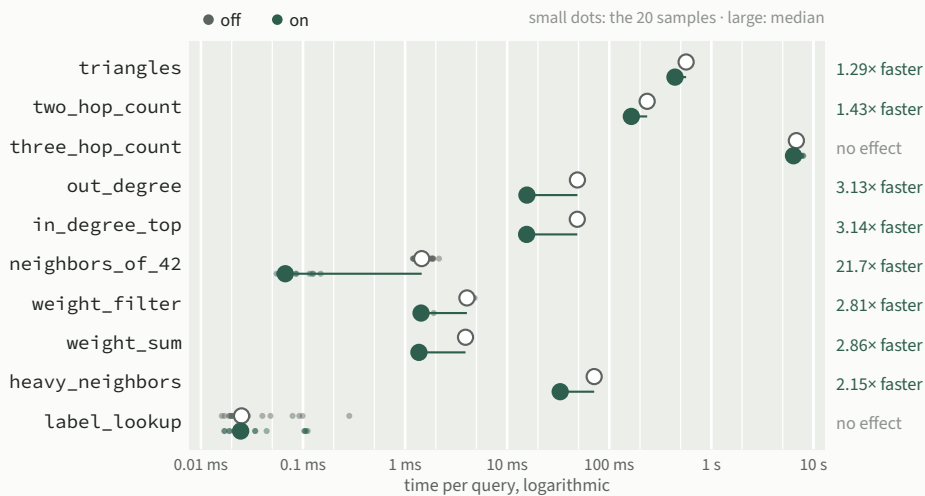


Figure 12: Small dots are the 20 samples of each arm, large markers the medians. “No effect”: the 95% interval of the ratio includes 1.

Correctness	643 tests pass at size 1 and at size 256. An equivalence test compares 11 queries, current and as-of, at sizes 1, 3, 16 and 256. That test found two silent write-path defects, both of which dropped or duplicated datoms during a split. Both are fixed.
Storage	The EAV key count fell from 45,953 to 224. Total bytes rose 8% from the length prefix per datom. Gains flatten past size 256.
Reading the chart	The scans and the point lookup gain the most. The joins gain less, and three_hop_count sits at the edge of no effect. Its interval touches 1, and the report counts it as unchanged.
Cost to integrate	A segment rewrite republishes the datoms of earlier transactions through the SlateDB change feed. This broke all 38 incremental-query tests until the CDC layer learned to keep only the newest transaction's datoms. Write amplification under random single-datom writes was not measured. Segment values must stay under SlateDB's key and value limits.

4.1 Where the time goes today

Chapter 3 reduces the number of keys a scan touches. Inside a segment, the datoms still sit one after another with all their components. A query that needs only the entity of each datom still reads and decodes its value, transaction id and flag. Nothing is compressed. An entity id takes nine bytes wherever it appears, even in a segment that holds two hundred consecutive datoms for the same entity.

4.2 The idea: store each component as its own array, then compress it

A *columnar* layout stores a segment as one array per component instead of one record per datom. The same seven datoms become an array of entities, an array of values, a bitmap of flags and an array of transaction ids. A reader that needs one component reads one array and skips the rest. Analytical databases such as DuckDB use this layout throughout for that reason. Each array holds values of one kind, and values of one kind compress well.

The compression is the standard pair from DuckDB and Parquet. *Frame of reference* subtracts the array's minimum from every value. A column of large integers becomes a column of small offsets. *Bit packing* then stores each offset in the smallest number of bits that fits the largest offset, instead of 64. A column of 256 sorted entity ids that span a range of 1000 needs 10 bits per entry, 320 bytes in total. The same column at 9 bytes per entry takes 2304 bytes.

row-major segment: 7 records $\times$ (9 + 9 + 1 + 8 bytes) = 189 bytes

E=1	V=2	+	tx1	E=1	V=3	+	tx1	E=2	V=3	+	tx1	E=2	V=4	+	tx1	E=3	V=4	+	tx1	E=4	V=1	+	tx2	E=5	V=1	+	tx1
-----	-----	---	-----	-----	-----	---	-----	-----	-----	---	-----	-----	-----	---	-----	-----	-----	---	-----	-----	-----	---	-----	-----	-----	---	-----

a reader wanting only entities must step through every record

columnar segment: one array per component, integers as frame of reference + bit packing

E	1	1	2	2	3	4	5	min 1 $\rightarrow$ offsets 0 0 1 1 2 3 4 $\rightarrow$ max 4 fits in 3 bits $\rightarrow$ 21 bits $\approx$ 3 bytes
V	2	3	3	4	4	1	1	min 1 $\rightarrow$ offsets 1 2 2 3 3 0 0 $\rightarrow$ max 3 fits in 2 bits $\rightarrow$ 14 bits $\approx$ 2 bytes
op	+	+	+	+	+	+	+	bitmap, 1 bit each $\rightarrow$ 1 byte
tx	1	1	1	1	1	2	1	min 1 $\rightarrow$ offsets 0 0 0 0 0 1 0 $\rightarrow$ 1 bit $\rightarrow$ 1 byte

about 7 bytes of data plus a small header per column; out_degree reads the E array only

the E array bit-packed at 3 bits per entry:

000	000	001	001	010	011	100
-----	-----	-----	-----	-----	-----	-----

Figure 13: The seven :g/to datoms as a row-major segment and as a columnar segment. Every column carries its own byte length, and a reader jumps over the columns it does not need. The benchmark's segments hold up to 1024 datoms. At that size the ratio is far larger than on seven.

4.3 Mapping onto the indexes

The layout applies to AEV and AVE, the two indexes graph queries use. The AE and AV indexes exist to enumerate distinct entities or distinct values for an attribute. In the new layout they are the first column of an AEV or AVE segment with duplicates collapsed. The experiment removes them and serves those scans from the first column, which is part of the storage reduction below. Temporal filtering reads the transaction column and resolves the visible version of each logical key from it. EAV and VAE keep the row layout.

4.4 Walkthrough: out_degree and three_hop_count

out_degree groups entities in AEV. The columnar scan reads the E column of each of the 55 segments for :g/to. It decodes 3 bytes per entry and never touches values or transactions. That gives the 0.30 $\times$ in the chart. three_hop_count performs sixteen million single-row seeks. Each seek decodes a whole segment's E and V columns to find one row. The decode cancels the key-count saving. In the first A/B, run while other

experiments were benchmarking, the query regressed to 1.25×. On the idle machine the ratio is 0.97 with an interval that includes 1, so the deep join neither gains nor loses. The row-major segments of chapter 3 show the same shape. Chapter 7 tests whether batched seeks turn this into a gain.

4.5 Results

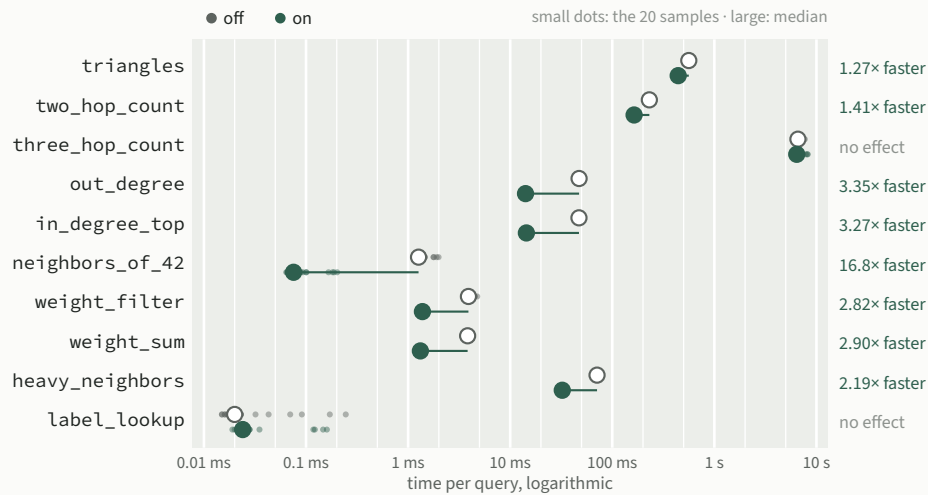


Figure 14: Small dots are the 20 samples of each arm, large markers the medians. “No effect”: the 95% interval of the ratio includes 1.

Correctness	645 tests pass with the toggle off. With the columnar layout forced on for the whole suite, 10 library tests fail. Each is a test-harness artifact, such as a test that writes raw row keys directly. The branch’s EXPERIMENT.md lists them. The equivalence test exposed an off-by-one in the first-column decoder that made every AE/AV scan panic. It is fixed.
Storage	At scale, bytes fell 61% and key count 69%. AEV for :g/to went from 46,173 keys to 55.
Point lookups	label_lookup is a 20 μs query, and its ratio is noise in every single experiment. Inside the full stack the columnar arm slows it from 0.02 to 0.03 ms with an interval that excludes 1: a string lookup on AVE now decodes a segment to find one key. The per-segment index proposed above would remove that cost.
Cost to integrate	The layout is a property of a database. It must be recorded in bootstrap metadata, not read from an environment variable. Compaction and updates rewrite whole segments. The change feed issue from chapter 3 applies unchanged.

5.1 Where the time goes today

Every graph query in the benchmark is a sequence of “follow the edges of x ” steps. Each step is an index seek: find the key range for entity x in AEV, iterate it, decode each datum, and hand the values back as Bytes. The join in chapter 1 performs that seek once per binding. The triangle query on the benchmark graph performs about 80,000 seeks to find 7,821 triangles. Every seek returns the same twenty or so neighbours that an earlier seek already read.

5.2 The idea: the graph as a matrix

A directed graph on n vertices can be written as an $n \times n$ *adjacency matrix* A . Entry (i, j) is 1 if there is an edge from i to j and 0 otherwise. Row i lists the out-neighbours of i . Column j lists the in-neighbours of j . The product of A with itself gives, at (i, k) , the number of two-step paths from i to k . The triangles are the entries of A^2 that also have a direct edge in A .

Almost all entries of a real adjacency matrix are 0. The benchmark graph has 4 million cells and 39,764 ones. A *sparse* representation stores only the ones.

The standard form is *compressed sparse row*, CSR. One array holds the column indices of all the ones, row after row. A second array of offsets says where each row starts. Row i is the slice between `offsets[i]` and `offsets[i+1]`. RedisGraph stored every relationship type as such a matrix with the GraphBLAS library and compiled Cypher pattern matching to sparse matrix algebra.

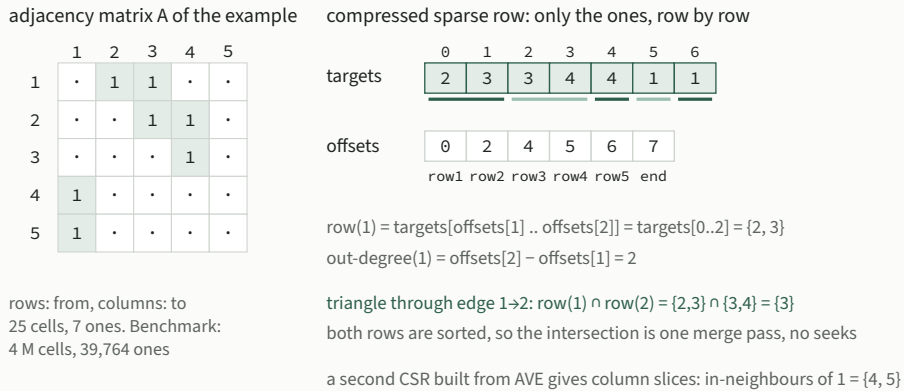


Figure 15: The example graph as a dense matrix and as CSR. Row lookups, degrees and intersections become slices of two integer arrays. The benchmark’s :g/to matrix takes 1.45 MB for both orientations, 36 bytes per edge, and about 40 ms to build.

5.3 Mapping onto Triplox

A reference attribute in Triplox is a relationship type. The AEV index for it, read in order, is the CSR targets array: entity after entity, each followed by its sorted values. One AEV scan builds the matrix and records where each entity’s run starts. The AVE index gives the transposed matrix the same way. Entity ids are integers. The row index of an entity is its position in a dense id map built during the same scan.

The matrix is built per reference attribute and per database basis, the transaction id a query reads at. The node caches it keyed by (attribute, basis). The matrix is a derived structure. The index remains the source of truth, and an as-of query at a different basis uses the existing path.

In the planner, an AdjacencyPattern replaces any triple pattern whose attribute is a reference type. It satisfies the same ExecPattern contract as a triple pattern but answers from the matrix. A bound entity gives a row slice, and a bound value gives a column slice. Both bound give a binary search, and neither bound

gives the full edge list. Scalar patterns are untouched. `[?b :g/weight ?w]` in `heavy_neighbors` still goes to the index.

5.4 Walkthrough: triangles

For each edge (a, b) in the full edge list, the third variable is `row(a) n row(b)`. Both rows are sorted slices of the targets array. The intersection is one merge pass over approximately forty integers. The whole query is about 40,000 merge passes over memory-resident arrays, with no allocation per binding. That gives the 0.03× in the chart. `out_degree` reads the offsets array once. `neighbors_of_42` is one slice.

5.5 Results

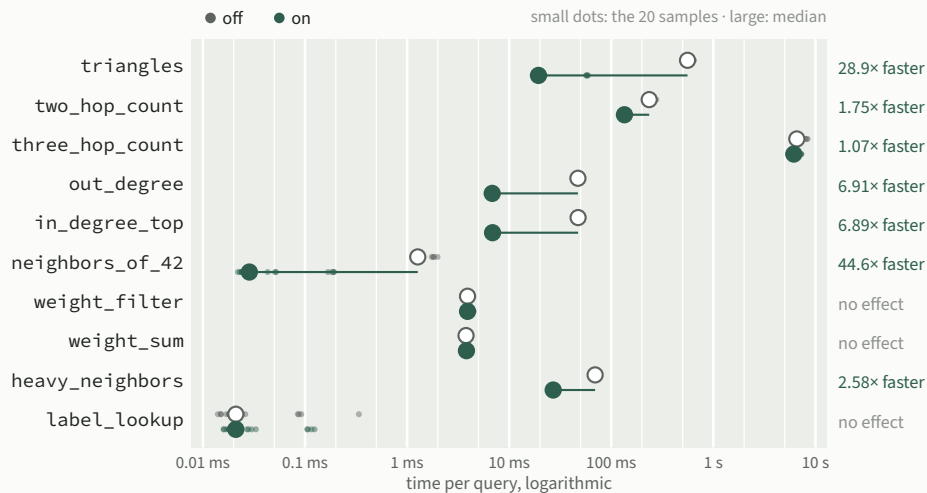


Figure 16: Small dots are the 20 samples of each arm, large markers the medians. “No effect”: the 95% interval of the ratio includes 1.

Correctness	642 tests pass on and off. The equivalence test covers 11 queries including a retraction.
Reading the chart	Queries on the reference attribute improve by 1.1× to 45×. The three scalar queries are unchanged, as they must be. <code>heavy_neighbors</code> improves less than the pure traversals because its scalar predicate still seeks the index once per candidate.
Side finding	In the first A/B, <code>heavy_neighbors</code> was 9.6× slower with the matrix on. The cause was a pre-existing planner defect, described in chapter 8. The matrix reported an exact candidate count and lost the proposer choice to an index estimate of zero. Two changes to candidate-set handling in the join fixed it without a change to the estimator.
Where this belongs	As a planner-selected path for chains and cycles of reference patterns, backed by a per-basis cache. Recursive rules, when they exist, have the same structure: each fixed-point iteration multiplies a frontier by the matrix. Memory is 36 bytes per edge. The cache needs a bound and an eviction policy before it can be on by default.

5.6 Matrix algebra `TRIPLOX_MATRIX_ALGEBRA=1`, on top of `TRIPLOX_ADJ_MATRIX=1`

The CSR cache above keeps the generic join and only changes how a reference pattern answers. On `three_hop_count` the join still enumerates approximately sixteen million (a, b, c, d) tuples before it counts them. The matrix only makes each lookup cheaper, and the result is 0.88×. RedisGraph never lists the paths. It keeps one number per vertex and updates all of them at once, one hop at a time. A second toggle adds that path for the query shapes where it provably gives the same answer as the join.

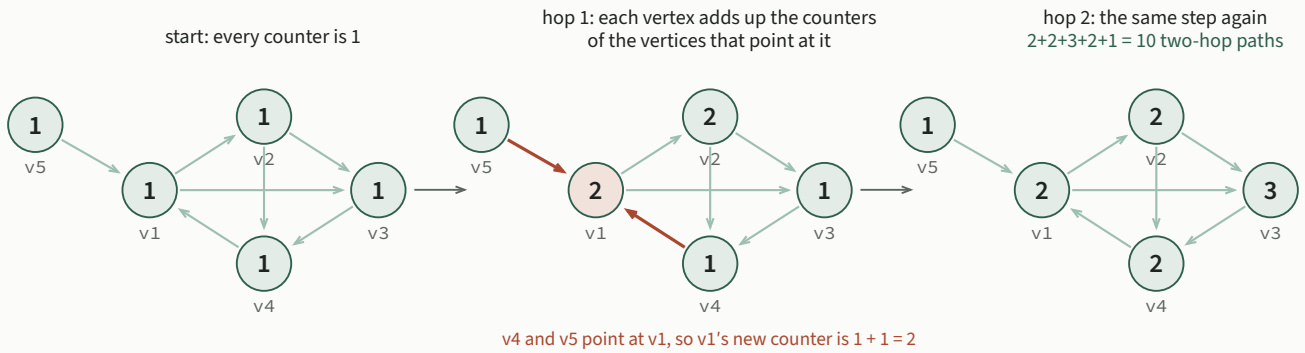


Figure 17: two_hop_count with one counter per vertex. The join answers the same query by listing all ten (a, b, c) paths.

The hop in figure 17 is a matrix operation. Figure 18 shows the same hop on the grid from section 5.2.

A: rows = from, columns = to						counters	
	v1	v2	v3	v4	v5		
v1	•	1	1	•	•	1	new counter of v1
v2	•	•	1	1	•	1	= the counters in the rows where column v1 has a 1
v3	•	•	•	1	•	1	= counter(v4) + counter(v5) = $1 + 1 = 2$
v4	1	•	•	•	•	1	one hop = this sum for every column
v5	1	•	•	•	•	1	new counters: 2 2 1 1 1. Only the 7 cells with a 1 do any work.

Figure 18: Hop 1 as a matrix operation, for the highlighted column.

The vocabulary for the two figures:

Adjacency matrix the grid A. A 1 at row i , column j means an edge from i to j .

Matrix-vector product one hop. Each new counter is a sum over one column of A. A is mostly zeros, so the product is *sparse* and costs one addition per edge.

$1^T \cdot A \cdot A \cdot A \cdot 1$ the three-hop count written out. Start with all ones, multiply by A three times, sum.

Semiring the rule for what “add” and “multiply” mean. count uses integers with + and $\times$. count-distinct only needs yes or no per vertex, so it uses “or” and “and”.

Bit kernels a row of 2000 yes/no values fits in 250 bytes, and the processor combines 128 of those bits in one instruction². The dense yes/no matrix is capped at 64 MB, and the sorted-list form takes over above that.

GraphBLAS a library interface for sparse products with a pluggable semiring³.

The prototype accepts a chain of reference patterns, $[?x0 : a ?x1] [?x1 : a ?x2] \dots$, with a single count or count-distinct in the find clause. The variables in the middle of the chain must not be used anywhere else in the same query. After a hop, a counter has merged every path that ends at its vertex, and the middle vertex of each path is lost. That is harmless when only the final count matters and wrong otherwise, so those queries run through the join.

The chain can start or end at a set fixed by other clauses, such as $[?a : g/id 42]$. That set comes from a small sub-query through the normal engine. The second shape is the triangle count, $[?a : a ?b] [?b : a ?c] [?a : a ?c]$ with a count, answered by combining the two-hop counters with the direct edges. Everything else returns “not handled” and runs as before. An equivalence test compares the two answers on three random graphs with retractions, at head and at an as-of basis: 17 queries asserted handled and equal, 14 asserted declined.

²SIMD, single instruction, multiple data. NEON is the SIMD instruction set on Arm processors, including the one this benchmark ran on.

³RedisGraph translated Cypher patterns into GraphBLAS calls. The prototype here writes the same operations directly in Rust over the CSR matrices of section 5.3.

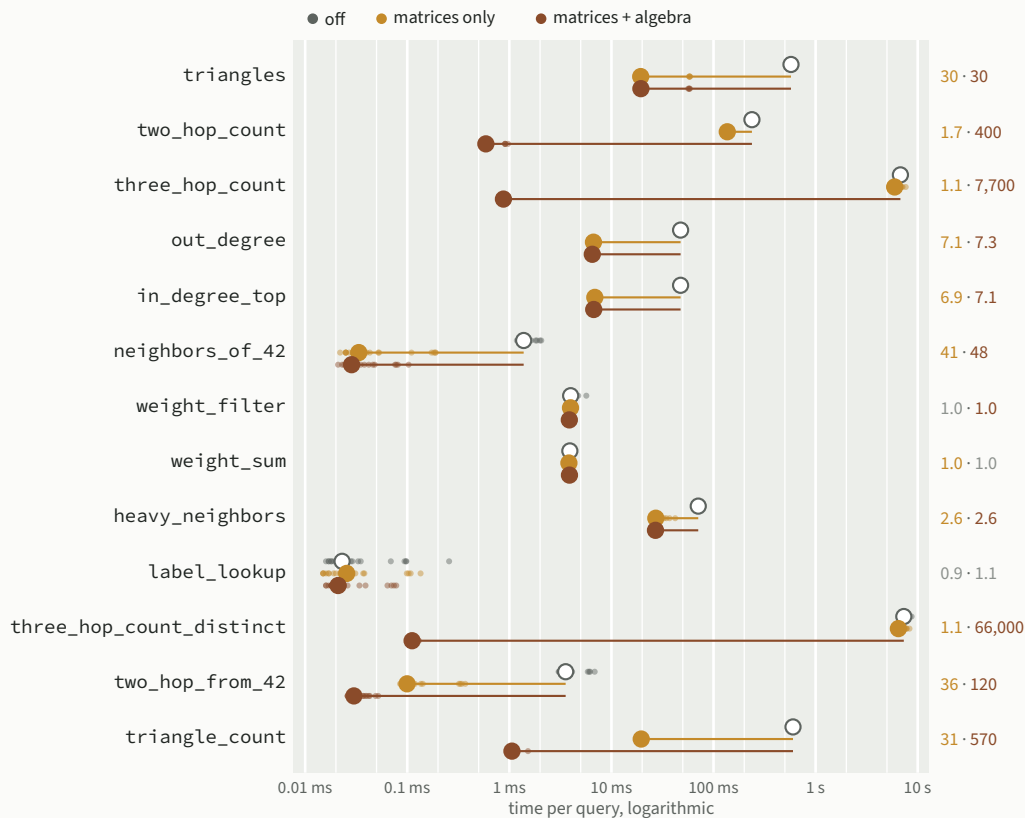


Figure 19: Small dots are the 20 samples of each arm, large markers the medians. Right: times faster than off per arm, grey when the 95% interval includes 1.

Correctness	Row counts identical across off, matrices and algebra arms for all 13 queries. The equivalence test covers 31 queries, including retractions and as-of bases. 648 tests pass with both toggles off, with the matrices alone, and with both on. Clippy and fmt are clean.
Reading the chart	The three aggregate-only queries the algebra handles fall by three to four orders of magnitude: three_hop_count 6.8 s to 0.9 ms, three_hop_count_distinct 7.3 s to 0.1 ms, triangle_count 600 ms to 1.1 ms, two_hop_count 238 ms to 0.6 ms. Every other query is unchanged from the matrices arm, as it must be: the tuple-returning triangles still enumerates its 7,821 rows.
What it does not do	Queries that return tuples. Chains whose middle variables are used elsewhere, or that use a variable twice. Aggregates other than count. Queries with inputs, :with, ordering or a limit. Recursive rules do not exist in Triplox yet. When they do, each fixed-point step is the same product.
Cost	One yes/no bit per pair of vertices, in each direction: 0.5 MB at 2000 vertices, capped at 64 MB. The CSR cache and its per-basis invalidation apply unchanged.

6.1 Where the time goes today

A range predicate such as `[(> ?w 900)]` runs after the scan. The pattern `[?e :g/weight ?w]` reads every weight datum, and the predicate discards 90% of them. An as-of query at an old basis reads every version of every key in its range and discards the ones written later. The index is sorted by key, and a query can skip key ranges. It has no summary of the *values* or *transaction ids* inside a key range, and it cannot skip on those.

6.2 The idea: minimum and maximum per block

DuckDB, and most column stores, keep the minimum and maximum of every block of a column. A predicate is checked against the summary first. If the block's maximum is below the predicate's lower bound, the block cannot contain a match, and the reader skips it. The summary is two values per block. It costs nothing when it does not help. Netezza introduced the technique under the name *zone map*, and Parquet stores the same statistics per row group.

Zone maps only work when the values in a block are clustered. If every block spans the full range of values, no block can be skipped. Column stores sort or partition data by the columns they expect predicates on for this reason.

6.3 Mapping onto the indexes

For each (index, attribute) prefix, the keys in key order are split into runs of 256. Each run records the minimum and maximum of the component outside the sort prefix, V for AEV and E for AVE. It also records the oldest and newest transaction id. The maps are built in memory by a scan of the prefix and cached per basis. A map built at basis S stays valid for any query at a basis $\leq S$. Keys are never deleted, and every later key carries a transaction id greater than S .

The planner pushes comparison predicates on a variable bound by exactly one pattern into that pattern's scan as value bounds. The temporal filter consults the transaction range and skips runs that are entirely newer than the query's basis.

Today: the scan reads every key and tests the predicate afterwards

16 keys of the AEV :g/weight index, in key order. The number in each cell is the weight. Query: w > 900.

7	42	60	88	120	135	190	240	500	560	610	700	910	930	960	999
---	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

16 keys read, 4 match

red underline = the key is read from storage · green cell = passes the predicate

With a zone map: one card per run of 4 keys holds the run's min and max, and the scan checks the card first

min 7 · max 88				min 120 · max 240				min 500 · max 700				min 910 · max 999			
7	42	60	88	120	135	190	240	500	560	610	700	910	930	960	999

4 keys read, 4 match

Three cards say "max below 900", so their 12 keys are never read. The map itself is 8 numbers.

The same 16 keys with the weights shuffled: every run now holds a large weight, and no run can be skipped

min 42 · max 910				min 120 · max 930				min 500 · max 960				min 7 · max 999			
910	42	60	88	120	930	190	240	500	960	610	700	7	135	560	999

16 keys read, 4 match

Zone maps only help when neighbouring keys hold similar values. In the benchmark, weight = id mod 1000 and keys are sorted by id, so they do.

Time travel: each card also holds the oldest and newest transaction in its run. Query: as of transaction 1

tx 2 to 3				tx 4 to 5				tx 6 to 7				tx 8 to 9			
2	2	3	3	4	4	5	5	6	6	7	7	8	8	9	9

0 keys read

The number in each cell is the transaction that wrote the key. Every card starts after transaction 1, so the reader skips all four runs without touching a key.

Figure 20: Zone maps on a slice of the weight index, before and after. A card costs two numbers per run and is checked before the run is read. On the benchmark, runs are 256 keys: the value predicate skipped 1230 of 2000 seeks, and the as-of queries skipped 155 of 155 runs. The shuffled row shows the limit of the technique: the map cannot skip a run that mixes small and large values. Transaction pruning does not have this limit, because Triplox never rewrites an old key, so transaction ids always rise along the key order.

6.4 Walkthrough: weight_filter and the as-of queries

weight_filter binds ?w through exactly one pattern. The planner converts > 900 into a lower bound on the scan of :g/weight in AEV. The scan consults the run summaries and skips runs whose maximum is at or below 900. On the benchmark data the weights rise with entity id, and 1230 of the 2000 per-entity seeks are pruned.

The experiment also added three as-of queries. Two read at a basis before the edges were loaded. There, the transaction range skips every run, 155 of 155, and the queries run 6 to 7× faster. The third reads at a basis in the middle of the load. Only the runs newer than that basis are skipped, and the gain is 1.4×. The first sample of label_lookup pays a 22 ms build of a map for :g/label that the query never uses. The median hides that cost, and the chart shows no effect.

6.5 Results

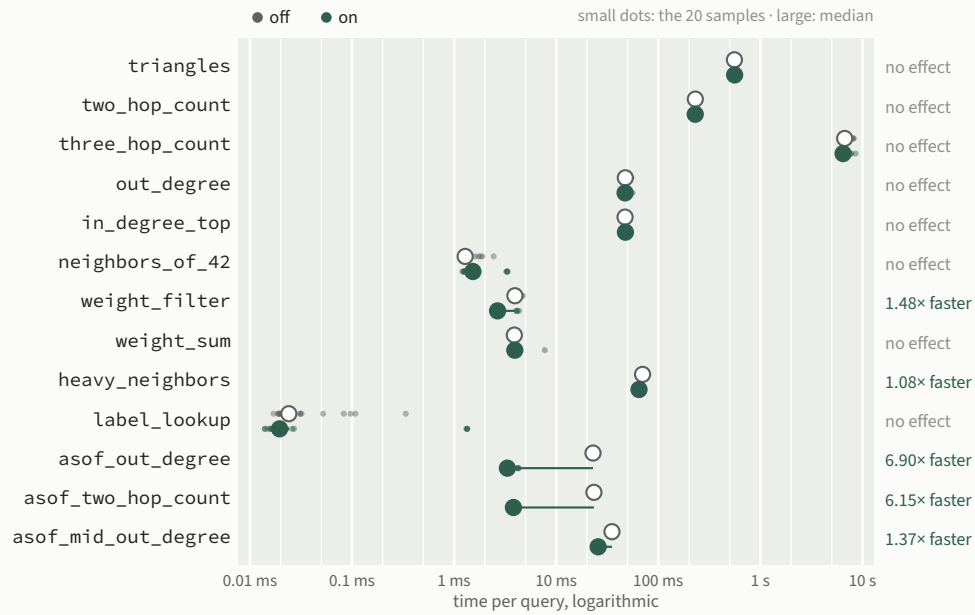


Figure 21: Small dots are the 20 samples of each arm, large markers the medians. “No effect”: the 95% interval of the ratio includes 1.

Correctness	647 tests pass, once normally and once with the toggle forced on for the whole suite. The equivalence test covers 10 queries × latest/as-of × cold/warm cache.
Reading the chart	Only rows with skipped runs show a real effect: <code>weight_filter</code> , <code>heavy_neighbors</code> , and the three as-of rows. <code>heavy_neighbors</code> gains 8% from the same value bound on <code>:g/weight</code> . Every other row has zero skips, and its ratio is noise.
Assessment	Transaction-range pruning is worth implementing, at approximately a week with incremental rebuild. Value pruning is not. A planner change that routes value-bounded patterns through AVE as a range scan serves the range predicate case better and needs no zone map. Counters in the experiment confirmed that the planner currently runs that query as 2000 per-entity seeks instead.
Defects in the prototype	Maps are built on every scan, including at head basis where they can never prune. The cache is unbounded, and each new basis rebuilds it with a full prefix scan.

7 Combinations

Three stacks were merged from the finished branches. Each was benchmarked with all toggles off, each toggle alone, and all toggles on. This attributes the contribution of each part instead of summing them. In every table the ratio is against the stack's own off arm. The “singles multiplied” comparison uses the single-toggle arms of the same run.

7.1 Vectorized join and row-major segments exp/combo-vec-seg

The two changes attack different costs: the join's intermediate rows and the storage's per-key overhead. The merge was one import block. All-on beats the product of the two single ratios on eight of ten queries. Each feature was partly hidden behind the other. The batched engine alone still paid an LSM lookup per datum, and segments alone still allocated a row per output.

neighbors_of_42 is the exception. A point lookup that returns 21 rows does not repay the batched engine's setup, and all-on is no better than segments alone on that query.

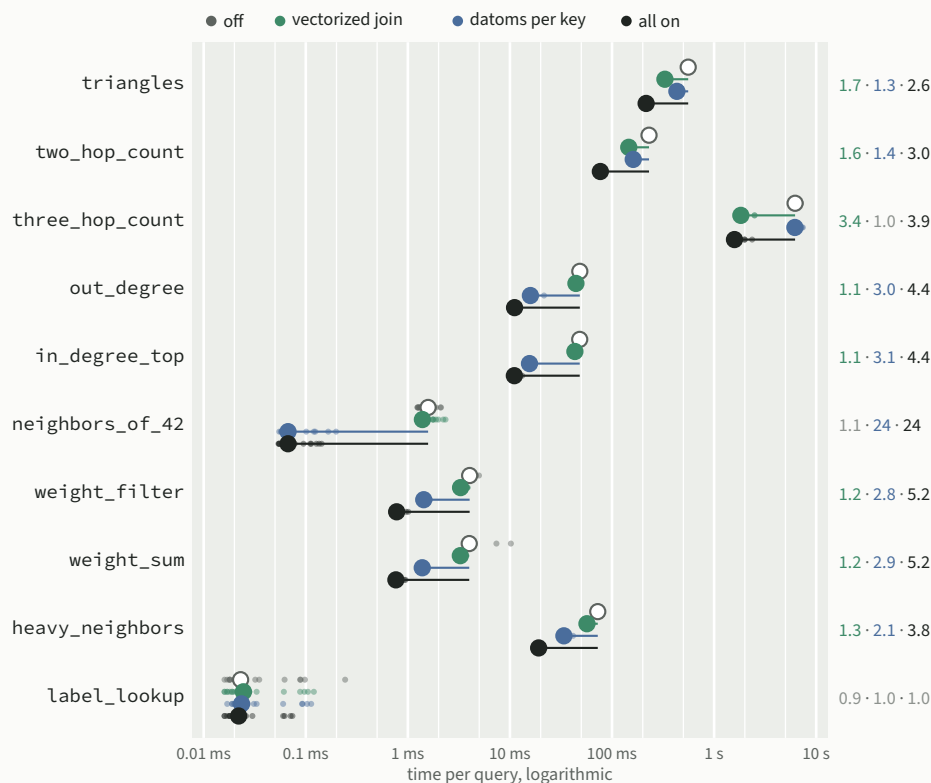


Figure 22: Small dots are the 20 samples of each arm, large markers the medians. Right: times faster than off per arm, grey when the 95% interval includes 1.

Correctness Row counts are identical to baseline in all four arms. 647 tests pass with both toggles off and both on. A new equivalence test compares the batched and row engines over segmented storage, the pairing neither branch covered.

Interaction three_hop_count gains nothing from segments alone, 1.00× with an interval that includes 1. On top of the batched engine the same segments give a further 14%, from 0.29× to 0.25×. The deep join is bound by row bookkeeping, not by key count, until the bookkeeping is removed.

7.2 Vectorized join and columnar segments exp/combo-vec-col

This stack answers the question left open in chapter 4. Columnar segments alone gave three_hop_count nothing, because each single-row seek decoded a whole segment. With the batched engine the same segments turn into a gain, from 0.28× to 0.25×. The mechanism is visible in the code, not only in the numbers.

The batched engine sorts each level’s bindings by key before it extends them, originally to avoid a BTreeMap. The segment iterator returns at once when it is already at or past a seek target. A run of ascending seeks therefore decodes each segment once. The row engine’s seek order is arbitrary, and that is the access pattern segments handle worst.

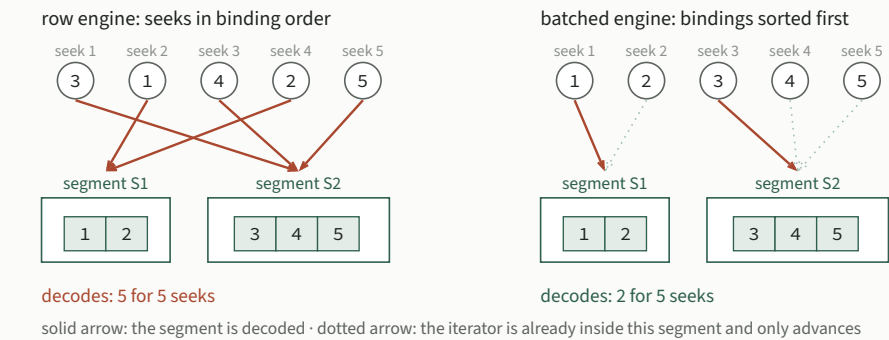


Figure 23: Why columnar segments help the deep join only under the batched engine. Each seek lands in the segment that owns its entity. Unsorted seeks bounce between segments and decode one per seek. Sorted seeks decode each segment once and advance within it. With 256-datom segments and about 20 seeks per segment, the decode cost per seek falls by an order of magnitude.

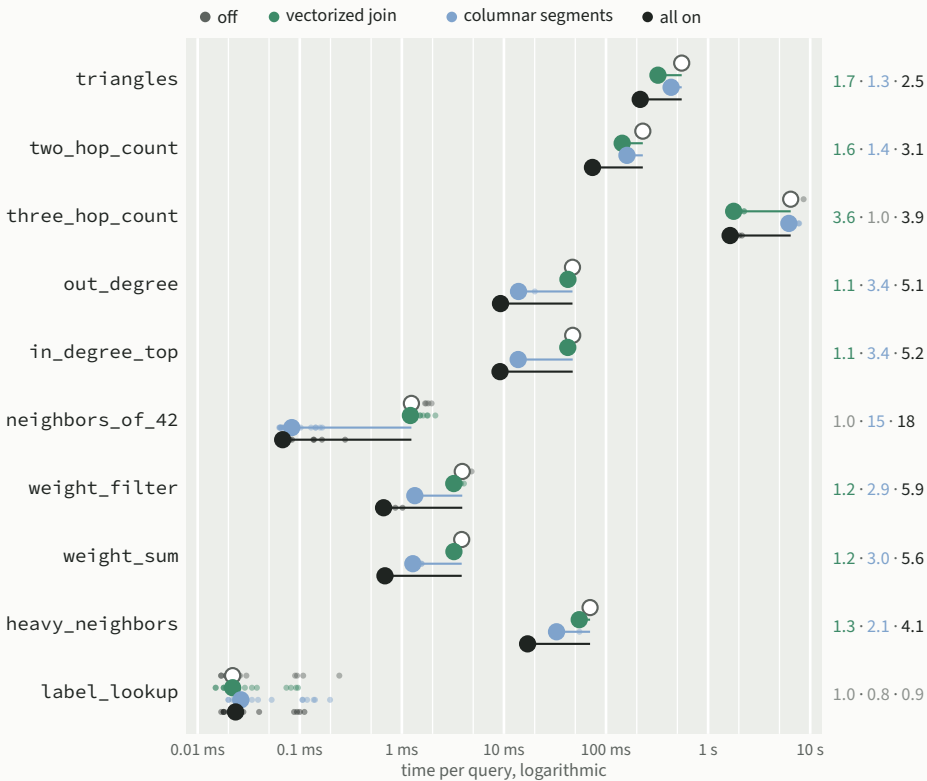


Figure 24: Small dots are the 20 samples of each arm, large markers the medians. Right: times faster than off per arm, grey when the 95% interval includes 1.

Correctness	Row counts are identical to baseline in all four arms. With toggles off, and with the batched join alone, all tests pass. Any configuration with the columnar layout fails 11 library tests: the 10 documented in chapter 4 plus the batched-versus-row equivalence test. That test seeds SlateDB with raw row keys and cannot run over segments. Under the columnar layout no test asserts that the two engines return identical rows. The matching bench row counts are the only evidence for that combination.
Interaction	Gains more than compose on every multi-level query: triangles 0.39× against a product of the two single ratios of 0.47×, two_hop_count 0.32× against 0.44×. The merge added no new code. The batched engine reads storage through the same two helper functions as the row engine, and a layout swap under them was sufficient.

7.3 The full stack exp/combo-full

This stack combines the vectorized join, columnar segments and adjacency matrices, and a sixth arm adds the matrix algebra of section 5.5 on top. The question is whether the matrices still pay once scans are cheap and the join is batched. The chart has six arms: everything off, each of the three toggles alone, the three together, and the three together plus the algebra toggle. The algebra is opt-in, so the “join + columnar + matrices” arm runs every query through the join, and the last arm differs from it only on the queries the algebra recognises.

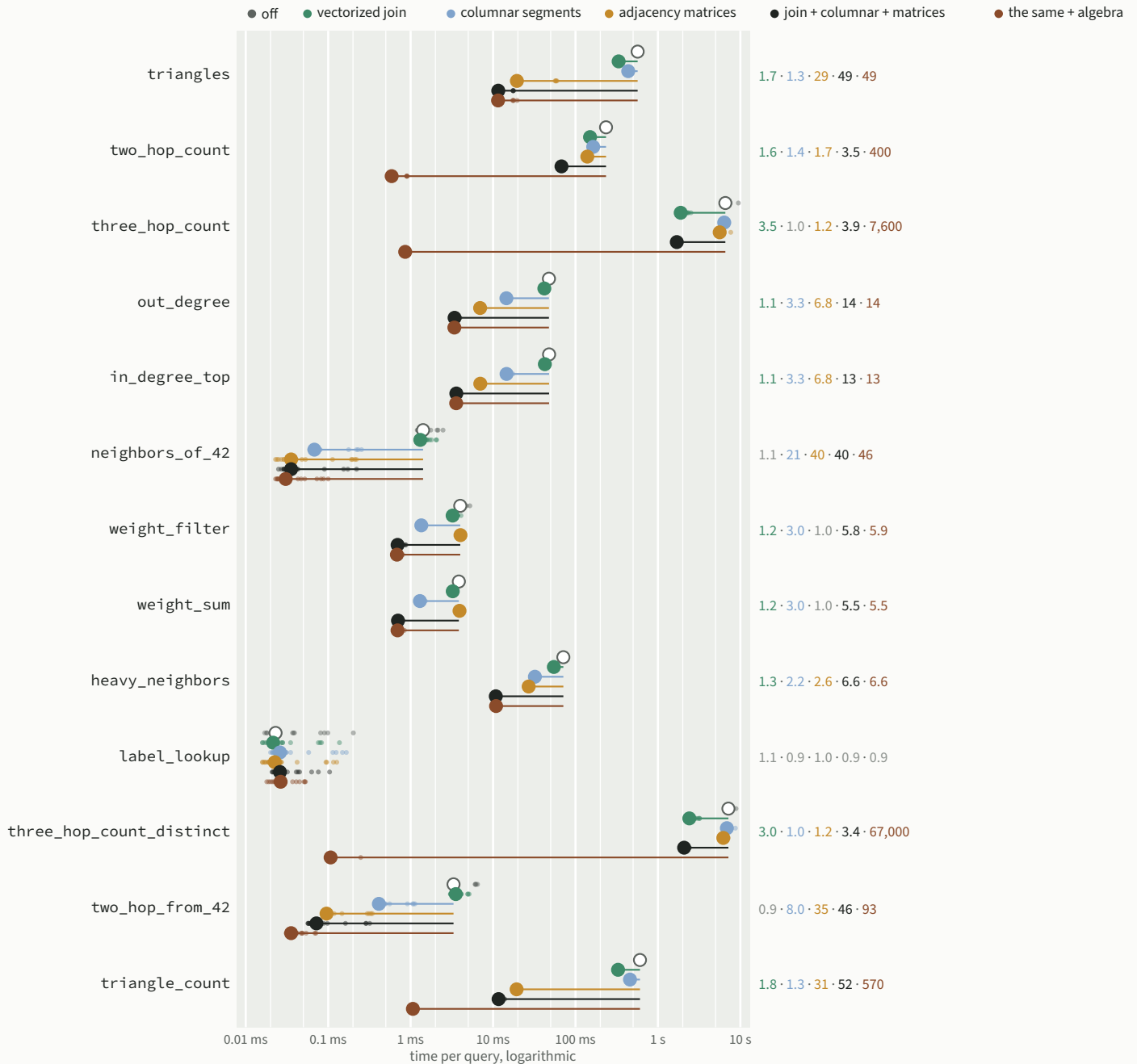


Figure 25: Small dots are the 20 samples of each arm, large markers the medians. Right: times faster than off per arm, grey when the 95% interval includes 1.

Correctness

Row counts are identical to baseline in every arm. Off, batched, adjacency, and adjacency with batched: all tests pass. Any configuration with the columnar layout fails the same 11 harness-artifact tests as the previous stack, no more. With the algebra merged in, the row layout passes 657 tests with all four toggles on, and the columnar layout fails the same 11.

Does the matrix still pay?

Yes. Columnar alone brings triangles to 0.78× and the batched join to 0.57×. The matrix alone brings it to 0.03×, and all three together to 0.02×, 12 ms against 563 ms. Cheaper

	scans and a faster join shrink the constant the matrix saves. But the matrix replaces a probe per candidate with one intersection, and no storage or layout change substitutes for that.
Which part does what	On the scans, <code>out_degree</code> and <code>in_degree_top</code> , the matrix gives 0.15× and the full stack 0.07×: the matrix answers from its offsets array, and columnar segments make building it cheaper. On the scalar queries the matrix does nothing, as it must, and the whole gain is the segments. On <code>three_hop_count</code> the batched join does all the work among the first three, 0.29× alone against 0.26× for the stack.
With the algebra	The algebra arm changes only the aggregate-only queries, and changes them completely: <code>three_hop_count</code> 6.6 s to 0.86 ms, <code>three_hop_count_distinct</code> 7.2 s to 0.11 ms, <code>two_hop_count</code> 235 ms to 0.59 ms, <code>triangle_count</code> 606 ms to 1.07 ms. Every other query is within noise of the all-three arm. The merge into this stack needed no code change beyond three conflict resolutions, and the row counts match across all six arms.
Integration	Merging the three prototypes exposed three interactions between their defaults, all fixed on the branch and listed in its <code>COMBINED.md</code> . One built a wrong matrix with no error. The adjacency equivalence test caught it.

8 Defects found

None of these was the object of an experiment. Each is fixed on the branch that met it and deserves an issue against main.

1. **Proposer selection ignores cost on uncompact data.** The generic join asks each pattern that can bind a variable for an estimated candidate count. The smallest estimate proposes. The estimate comes from SlateDB range statistics. These return 0 for every prefix while data is resident in the memtable or write-ahead log. With every estimate at 0, ties go to pattern order. The defect surfaced when the adjacency matrix reported an exact count of 7 to 37 and lost to an estimate of 0. The `:g/weight` pattern then proposed all 2000 entities per row and materialized four million rows for `heavy_neighbors`. Main has this defect today on any dataset that has not been compacted.
2. **Integer index keys sort in descending order.** The codec encodes longs as `value ^ i64::MAX`. This reverses order over the whole range, while doubles and strings sort ascending. The comment in the codec describes the encoding as order-preserving. No current query path depends on ascending order. A value range scan on AVE, which chapter 6 recommends, must account for the direction.
3. **Segment rewrites replay earlier datoms through the change feed.** A segment rewrite republishes the datoms of older transactions. Any layout that stores several datoms per key does this. The incremental engine subscribes to that feed. It double-counted every replayed datom until the CDC layer was changed to keep only datoms from the newest transaction in a batch.
4. **Two split defects and one decode off-by-one** in the segment prototypes themselves. An on/off equivalence test caught each one before benchmarking. They are listed because they show what the equivalence tests are for.

9 Memory, writes and the object store

The benchmark ran on an in-memory node and measured only time. Nobody measured memory use or object-store traffic. The figure below gives both for each technique, worked out from the code and from the benchmark's sizes. Only the two disk sizes marked as measured come from a run.

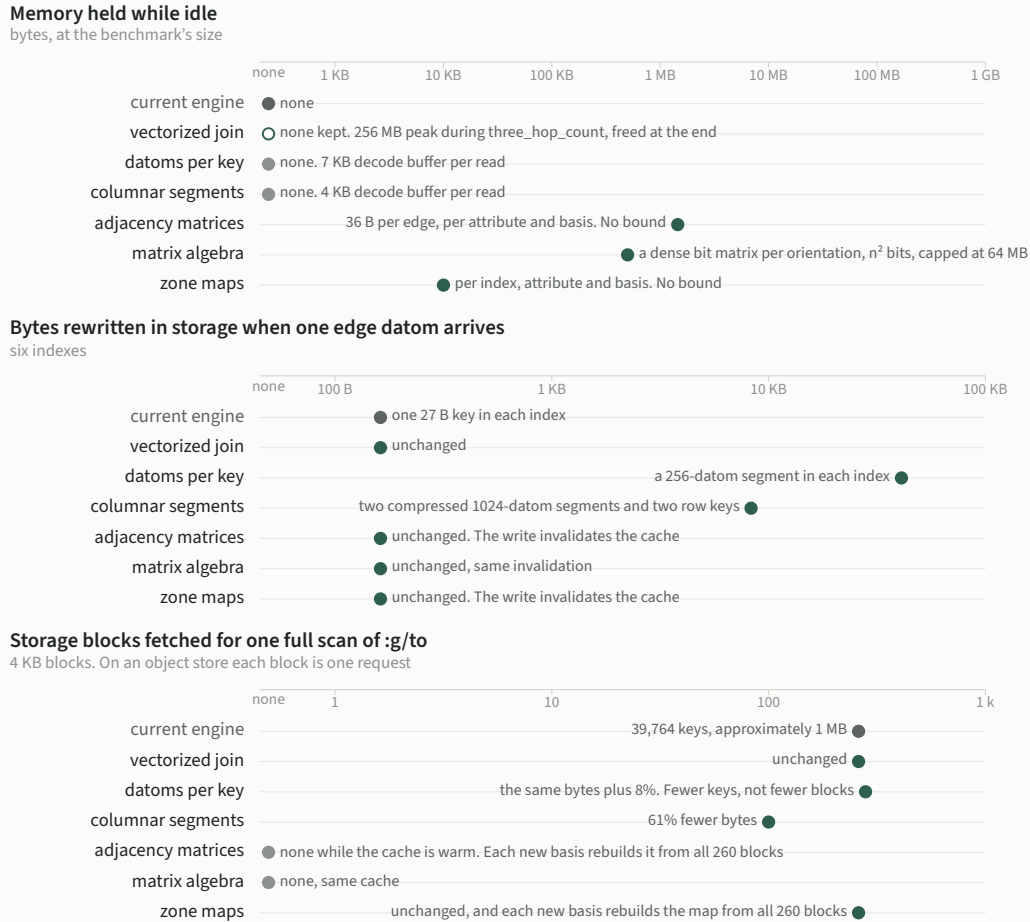


Figure 26: Block counts assume SlateDB's 4 KB blocks and are estimates. Hollow marker: a short-lived peak, not memory that stays allocated.

Three things follow for an object store, where every block is a request that costs tens of milliseconds and a fraction of a cent. The key-count reduction of chapter 3 is a CPU gain, not a fetch gain: the same bytes sit in the same number of blocks. Only the byte reduction of the columnar layout fetches fewer blocks, and both layouts write more, at roughly twelve times the price per request that a read costs. The vectorized join saves CPU and nothing else, which is a smaller share of a query that waits on the network. And the caches of chapters 5 and 6 are the strongest lever or the worst cost, depending on one thing: a warm matrix answers a traversal with zero fetches, while a rebuild per basis reads every block of the attribute. On an object store they pay only if the change feed maintains them.

10 Conclusions

Do these, in this order. Against an object store, items 2 and 3 swap: a cache that removes fetches is worth more than a layout that shrinks them, once the change feed maintains it.

1 Vectorized join

Every query with more than one pattern runs 1.1× to 3.5× faster. It touches the query engine and nothing else.

2 Segments, columnar

Many datoms per key makes scans and point lookups 3× to 22× faster in memory. Against an object store only the columnar layout's 61% byte reduction fetches fewer blocks, and both layouts write more, so build the columnar one and measure the write cost first. It alters the storage format and what the change feed carries, so design it before writing it, and give point lookups a per-segment index.

3 Matrices and algebra

Aggregate-only graph counts fall from seconds to a millisecond, and a warm matrix answers a traversal with no storage fetch at all, which is the largest possible gain against an object store. Worth it only if graph queries exist, and only once the matrices are fed from the change feed: rebuilt per basis, they read the whole attribute per transaction and cost more than they save.

4 Zone maps

1.4× to 7× on time-travel queries, nothing elsewhere. Value pruning depends on the data, not on Triplox.

The benchmark flatters items 3 and 4. It is one uniform integer graph with one reference attribute, and seven of its ten queries traverse that attribute. Real workloads are mostly strings, small tables and scalar expressions, with few joins. Items 1 and 2 cut costs every query pays; the others need a realistic workload before they are worth more than an experiment, and string handling was not measured at all.

Two defects in main are harmless today and stop being harmless later. The planner's estimate of zero on uncompact data makes plans follow clause order, which fails the moment one pattern reports a true count, as the matrices do. Integer keys sort in descending order, which fails the moment a value-range scan on AVE exists.

Wide scalar scans stay far from column-store speed. After every technique, `weight_sum` still spends approximately 0.3 μs per value. That comparison is an estimate: no DuckDB run was made.

11 Appendix: full statistics

Each table lists, per query, the sample count and the median of each arm with its 95% bootstrap interval. It then lists the ratio of medians with its interval and the two-sided Mann-Whitney p-value. Combination stacks list the ratio of each arm to the off arm.

11.1 Vectorized join

query	n	off ms	95% CI	on ms	95% CI	on / off	95% CI	p
triangles	20	559	[555, 568]	333	[325, 336]	0.60×	[0.58, 0.60]	<0.001
two_hop_count	20	231	[228, 231]	144	[143, 145]	0.62×	[0.62, 0.63]	<0.001
three_hop_count	20	6613	[6569, 7591]	1876	[1783, 2073]	0.28×	[0.25, 0.31]	<0.001
out_degree	20	47.4	[47, 48.1]	42.2	[41.9, 42.5]	0.89×	[0.88, 0.90]	<0.001
in_degree_top	20	47	[46.9, 47.4]	42.3	[42, 42.4]	0.90×	[0.89, 0.90]	<0.001
neighbors_of_42	20	1.31	[1.19, 1.56]	1.33	[1.24, 1.52]	1.02×	[0.84, 1.21]	0.365
weight_filter	20	4.09	[3.97, 4.21]	3.24	[3.17, 3.32]	0.79×	[0.77, 0.82]	<0.001
weight_sum	20	3.85	[3.81, 3.98]	3.23	[3.2, 3.26]	0.84×	[0.81, 0.85]	<0.001
heavy_neighbors	20	70.8	[69.8, 72.1]	54.7	[54.3, 55.2]	0.77×	[0.76, 0.79]	<0.001
label_lookup	20	0.02	[0.02, 0.04]	0.02	[0.02, 0.02]	0.81×	[0.50, 1.11]	0.147

11.2 Multiple datoms per key

query	n	off ms	95% CI	on ms	95% CI	on / off	95% CI	p
triangles	20	566	[564, 573]	440	[437, 442]	0.78×	[0.77, 0.78]	<0.001
two_hop_count	20	235	[233, 239]	164	[162, 165]	0.70×	[0.69, 0.71]	<0.001
three_hop_count	20	6804	[6537, 7018]	6378	[6115, 6735]	0.94×	[0.89, 1.00]	0.053
out_degree	20	48.7	[48.3, 49.9]	15.6	[15.3, 15.8]	0.32×	[0.31, 0.33]	<0.001
in_degree_top	20	48.6	[48.4, 49]	15.5	[15.3, 15.6]	0.32×	[0.31, 0.32]	<0.001
neighbors_of_42	20	1.45	[1.24, 1.77]	0.07	[0.06, 0.08]	0.05×	[0.04, 0.06]	<0.001
weight_filter	20	4.03	[3.98, 4.08]	1.43	[1.42, 1.47]	0.36×	[0.35, 0.36]	<0.001
weight_sum	20	3.91	[3.84, 4.02]	1.37	[1.35, 1.39]	0.35×	[0.34, 0.36]	<0.001
heavy_neighbors	20	71.1	[70.9, 71.8]	33	[32.8, 33.6]	0.46×	[0.46, 0.47]	<0.001
label_lookup	20	0.03	[0.02, 0.03]	0.02	[0.02, 0.03]	0.98×	[0.69, 1.50]	0.755

11.3 Columnar segments

query	n	off ms	95% CI	on ms	95% CI	on / off	95% CI	p
triangles	20	562	[554, 564]	442	[438, 445]	0.79×	[0.78, 0.80]	<0.001
two_hop_count	20	231	[228, 232]	164	[161, 165]	0.71×	[0.70, 0.72]	<0.001
three_hop_count	20	6584	[6456, 6791]	6411	[6331, 6927]	0.97×	[0.94, 1.06]	0.675
out_degree	20	47.3	[46.9, 47.6]	14.1	[14, 14.4]	0.30×	[0.29, 0.31]	<0.001
in_degree_top	20	47.2	[46.7, 47.6]	14.4	[14.3, 14.5]	0.31×	[0.30, 0.31]	<0.001
neighbors_of_42	20	1.27	[1.24, 1.39]	0.08	[0.07, 0.1]	0.06×	[0.05, 0.08]	<0.001
weight_filter	20	3.9	[3.85, 3.94]	1.38	[1.37, 1.41]	0.35×	[0.35, 0.36]	<0.001
weight_sum	20	3.83	[3.81, 3.87]	1.32	[1.29, 1.34]	0.34×	[0.33, 0.35]	<0.001
heavy_neighbors	20	70.9	[70.2, 71.9]	32.4	[32.2, 32.7]	0.46×	[0.45, 0.46]	<0.001
label_lookup	20	0.02	[0.02, 0.03]	0.02	[0.02, 0.03]	1.20×	[0.87, 1.49]	0.056

11.4 Sparse adjacency matrices

query	n	off ms	95% CI	on ms	95% CI	on / off	95% CI	p
triangles	20	558	[556, 567]	19.4	[19.3, 20.9]	0.03×	[0.03, 0.04]	<0.001
two_hop_count	20	235	[233, 236]	135	[134, 138]	0.57×	[0.57, 0.59]	<0.001
three_hop_count	20	6568	[6428, 6766]	6139	[5930, 6483]	0.93×	[0.89, 0.99]	0.006
out_degree	20	47.1	[46.8, 48]	6.82	[6.77, 6.92]	0.14×	[0.14, 0.15]	<0.001
in_degree_top	20	47.3	[47, 47.5]	6.87	[6.81, 7]	0.15×	[0.14, 0.15]	<0.001
neighbors_of_42	20	1.27	[1.24, 1.35]	0.03	[0.03, 0.05]	0.02×	[0.02, 0.04]	<0.001
weight_filter	20	3.91	[3.88, 4]	3.91	[3.84, 4]	1.00×	[0.97, 1.03]	0.588
weight_sum	20	3.78	[3.75, 3.86]	3.81	[3.77, 3.87]	1.01×	[0.98, 1.03]	0.449
heavy_neighbors	20	69.5	[69.1, 70.1]	27	[26.8, 27.2]	0.39×	[0.38, 0.39]	<0.001
label_lookup	20	0.02	[0.02, 0.02]	0.02	[0.02, 0.03]	1.00×	[0.81, 1.46]	0.616

11.5 Matrix algebra

query	off ms	adj / off	algebra / off
triangles	574	0.03× [0.03, 0.03]	0.03× [0.03, 0.03]
two_hop_count	238	0.57× [0.56, 0.58]	0.00× [0.00, 0.00]
three_hop_count	6771	0.88× [0.86, 0.91]	0.00× [0.00, 0.00]
out_degree	47.6	0.14× [0.14, 0.14]	0.14× [0.13, 0.14]
in_degree_top	47.6	0.14× [0.14, 0.15]	0.14× [0.14, 0.14]
neighbors_of_42	1.38	0.02× [0.02, 0.04]	0.02× [0.02, 0.03]
weight_filter	3.99	1.00× [0.95, 1.06]	0.97× [0.93, 0.99]
weight_sum	3.91	0.98× [0.97, 0.99]	0.99× [0.98, 1.01]
heavy_neighbors	70.9	0.38× [0.38, 0.39]	0.38× [0.38, 0.39]
label_lookup	0.02	1.11× [0.61, 1.50]	0.91× [0.59, 1.26]
three_hop_count_distinct	7310	0.88× [0.85, 0.91]	0.00× [0.00, 0.00]
two_hop_from_42	3.56	0.03× [0.02, 0.04]	0.01× [0.01, 0.01]
triangle_count	600	0.03× [0.03, 0.03]	0.00× [0.00, 0.00]

11.6 Zone maps

query	n	off ms	95% CI	on ms	95% CI	on / off	95% CI	p
triangles	20	556	[553, 562]	559	[556, 568]	1.01×	[0.99, 1.02]	0.298
two_hop_count	20	230	[230, 233]	230	[229, 232]	1.00×	[0.98, 1.01]	0.394
three_hop_count	20	6690	[6518, 6798]	6454	[6373, 6716]	0.96×	[0.95, 1.01]	0.172
out_degree	20	47.5	[47.2, 48.2]	47.2	[47, 47.7]	0.99×	[0.98, 1.01]	0.425
in_degree_top	20	47.2	[47, 47.5]	47.6	[47.3, 47.8]	1.01×	[1.00, 1.02]	0.133
neighbors_of_42	20	1.28	[1.22, 1.38]	1.53	[1.32, 1.59]	1.19×	[1.00, 1.27]	0.05
weight_filter	20	3.94	[3.9, 4.05]	2.66	[2.63, 2.79]	0.68×	[0.66, 0.71]	<0.001
weight_sum	20	3.89	[3.83, 3.94]	3.93	[3.83, 3.99]	1.01×	[0.98, 1.03]	0.473
heavy_neighbors	20	70.1	[69.2, 70.8]	64.7	[64.1, 65.7]	0.92×	[0.91, 0.94]	<0.001
label_lookup	20	0.02	[0.02, 0.03]	0.02	[0.02, 0.02]	0.81×	[0.55, 1.08]	0.104
asof_out_degree	20	22.9	[22.8, 23.1]	3.32	[3.27, 3.52]	0.14×	[0.14, 0.15]	<0.001
asof_two_hop_count	20	23.4	[23.3, 23.4]	3.8	[3.72, 3.86]	0.16×	[0.16, 0.17]	<0.001
asof_mid_out_degree	20	35.1	[34.9, 35.5]	25.7	[25.5, 25.9]	0.73×	[0.72, 0.74]	<0.001

11.7 Vectorized join and row-major segments

query	off ms	batched / off	segments / off	all / off
triangles	557	0.59× [0.58, 0.60]	0.77× [0.77, 0.78]	0.39× [0.39, 0.39]
two_hop_count	230	0.63× [0.62, 0.64]	0.70× [0.69, 0.72]	0.33× [0.33, 0.34]
three_hop_count	6212	0.29× [0.28, 0.32]	1.00× [0.97, 1.04]	0.25× [0.24, 0.27]

query	off ms	batched / off	segments / off	all / off
out_degree	48.3	0.92× [0.90, 0.93]	0.33× [0.32, 0.33]	0.23× [0.22, 0.23]
in_degree_top	48.2	0.90× [0.88, 0.91]	0.32× [0.32, 0.33]	0.23× [0.23, 0.23]
neighbors_of_42	1.58	0.88× [0.75, 1.19]	0.04× [0.04, 0.05]	0.04× [0.04, 0.06]
weight_filter	4.03	0.82× [0.78, 0.84]	0.36× [0.34, 0.37]	0.19× [0.18, 0.20]
weight_sum	3.99	0.82× [0.79, 0.84]	0.35× [0.34, 0.35]	0.19× [0.18, 0.20]
heavy_neighbors	72.5	0.78× [0.77, 0.80]	0.47× [0.46, 0.48]	0.26× [0.26, 0.27]
label_lookup	0.02	1.07× [0.66, 1.43]	1.02× [0.69, 1.50]	0.96× [0.63, 1.18]

11.8 Vectorized join and columnar segments

query	off ms	batched / off	columnar / off	all / off
triangles	551	0.59× [0.58, 0.59]	0.79× [0.78, 0.80]	0.39× [0.39, 0.39]
two_hop_count	229	0.63× [0.62, 0.64]	0.70× [0.69, 0.71]	0.32× [0.32, 0.32]
three_hop_count	6441	0.28× [0.27, 0.30]	0.96× [0.93, 1.02]	0.25× [0.25, 0.27]
out_degree	46.9	0.90× [0.89, 0.91]	0.30× [0.29, 0.30]	0.20× [0.19, 0.20]
in_degree_top	47.1	0.90× [0.89, 0.90]	0.29× [0.29, 0.30]	0.19× [0.19, 0.20]
neighbors_of_42	1.24	0.98× [0.93, 1.15]	0.07× [0.05, 0.08]	0.05× [0.05, 0.06]
weight_filter	3.91	0.83× [0.81, 0.85]	0.34× [0.33, 0.35]	0.17× [0.17, 0.17]
weight_sum	3.86	0.84× [0.83, 0.85]	0.33× [0.33, 0.34]	0.18× [0.17, 0.18]
heavy_neighbors	69.8	0.78× [0.78, 0.79]	0.47× [0.46, 0.47]	0.24× [0.24, 0.25]
label_lookup	0.02	1.00× [0.79, 1.54]	1.20× [0.98, 3.68]	1.07× [0.86, 1.32]

11.9 The full stack

query	off ms	batched / off	columnar / off	adj / off	all / off	algebra / off
triangles	567	0.59× [0.57, 0.60]	0.77× [0.76, 0.78]	0.03× [0.03, 0.04]	0.02× [0.02, 0.02]	0.02× [0.02, 0.02]
two_hop_count	235	0.64× [0.63, 0.65]	0.70× [0.69, 0.71]	0.59× [0.58, 0.60]	0.29× [0.29, 0.29]	0.00× [0.00, 0.00]
three_hop_count	6573	0.29× [0.28, 0.30]	0.97× [0.94, 1.01]	0.85× [0.83, 0.89]	0.26× [0.25, 0.27]	0.00× [0.00, 0.00]
out_degree	47.6	0.88× [0.87, 0.89]	0.30× [0.29, 0.31]	0.15× [0.14, 0.15]	0.07× [0.07, 0.08]	0.07× [0.07, 0.07]
in_degree_top	47.8	0.89× [0.87, 0.90]	0.30× [0.30, 0.31]	0.15× [0.14, 0.15]	0.07× [0.07, 0.08]	0.07× [0.07, 0.08]
neighbors_of_42	1.42	0.93× [0.84, 1.11]	0.05× [0.04, 0.05]	0.03× [0.02, 0.04]	0.03× [0.02, 0.03]	0.02× [0.02, 0.03]
weight_filter	4	0.81× [0.78, 0.85]	0.34× [0.32, 0.35]	1.01× [0.96, 1.06]	0.17× [0.17, 0.19]	0.17× [0.16, 0.18]
weight_sum	3.85	0.84× [0.81, 0.86]	0.34× [0.32, 0.34]	1.02× [0.98, 1.05]	0.18× [0.18, 0.19]	0.18× [0.17, 0.19]
heavy_neighbors	71.2	0.77× [0.76, 0.78]	0.45× [0.45, 0.46]	0.38× [0.38, 0.39]	0.15× [0.15, 0.15]	0.15× [0.15, 0.15]
label_lookup	0.02	0.93× [0.57, 1.27]	1.13× [0.70, 1.50]	0.98× [0.60, 1.23]	1.13× [0.69, 1.59]	1.15× [0.70, 1.45]
three_hop_count_distinct	7161	0.34× [0.32, 0.37]	0.96× [0.93, 0.99]	0.87× [0.85, 0.90]	0.29× [0.29, 0.30]	0.00× [0.00, 0.00]
two_hop_from_42	3.32	1.06× [0.96, 1.24]	0.12× [0.12, 0.15]	0.03× [0.03, 0.03]	0.02× [0.02, 0.03]	0.01× [0.01, 0.01]
triangle_count	606	0.54× [0.53, 0.56]	0.76× [0.74, 0.78]	0.03× [0.03, 0.03]	0.02× [0.02, 0.02]	0.00× [0.00, 0.00]

Each experiment branch contains an EXPERIMENT.md with hook points, negative results and integration cost, and a PROGRESS.md hand-off. Result JSON for every A/B run is retained alongside this report.